

Artificial Intelligence in Poker: A Comparative Analysis of Decision-Making Using Search Algorithms

Anna Brunkhorst

Surabhi Keesara

Nancy Onyejiaka

Northeastern University, Boston, MA

Abstract

This paper investigates the application of three distinct artificial intelligence algorithms in the complex and strategic environment of Texas Hold'em poker. We implemented agents for each of Expectiminimax, Monte Carlo Tree Search (MCTS), and Deep Q-Learning and conducted comparative analyses against simple poker bots. The study evaluates these algorithms' performance in terms of win rates and net winnings, revealing their respective strengths and adaptability in a stochastic and partially observable domain. Our results demonstrate that all three algorithms consistently outperform these simple bots (after some training in the case of Deep Q). MCTS showed superior adaptability and a high win rate. Deep Q-Learning exhibited significant potential for long-term strategic development, despite requiring a larger number of episodes to consistently perform well.

The repository for this study can be found [here](#), as well as in its citation in our references.

Introduction

Texas Hold'em is a popular variant of poker that combines strategy, psychology, and probability. This combination of attributes makes the game a prime example of a complex, partially observable, and stochastic game environment that presents unique challenges from an artificial intelligence perspective.

Each player begins the game with two private cards, known as hole cards, and aims to make the best five-card hand using these and five community cards dealt in three phases. The first three community cards are revealed in the flop, the fourth card is revealed in the turn, and the final card is revealed in the river phase. Players have the option to check, call, raise, or fold during the betting rounds that happen after the initial dealing of personal cards, post-flop, post-turn, and post-river. The

winning player is the last player standing who manages to create the strongest hand using five of the seven available cards.

The game's partially observable nature stems from the fact that players can only view their cards and the revealed community cards. Its stochastic nature is evident in the random dealing of cards. Players engage in strategic decisions: betting, calling, raising, or folding, showcasing the game's dynamic and uncertain nature. These characteristics make Texas Hold'em an ideal domain for AI algorithms designed for probabilistic reasoning and strategic decision-making in environments with incomplete and unpredictable information.

This study aims to analyze how different algorithms navigate these challenges. We implemented Expectiminimax, an adversarial tree search algorithm that combines minimax decision-making with expected value calculations; Monte Carlo Tree Search, known for its effectiveness in large decision spaces and adaptability in complex game scenarios; and Deep Q-Learning, a reinforcement learning approach that employs deep neural networks for long-term strategic planning. Each implementation was rigorously tested against predefined poker bots, providing insights into their decision-making processes, efficiency, and adaptability.

By comparing these algorithms in the dynamic and uncertain environment of Texas Hold'em poker, we seek to contribute to the understanding of AI's potential in strategic decision-making under uncertainty. This research not only furthers the application of AI in games but also provides valuable insights into AI's broader capabilities in real-world situations that require probabilistic reasoning and adaptive strategy development.

Hand value was calculated by generating possible 5-card hands from the given cards, assigning a numerical value/score to each possible hand based on its rank, and returning the average (if not all cards dealt) or

highest (if all cards dealt) of these values. Pot size and player chips were both fetched directly from the game state.

Background

Problem Formalization

As a game featuring partially observable information, stochastic elements, and strategic complexities, poker offers an intricate domain. The following is a formalization of poker as a search problem.

- **State Representation** — **S**: $S = \{ \text{players, deck, phase, communityCards, agentHand, potSize, actionHistory} \}$
- **Players** — **P**: $P = \{ \text{AI, Randomized} \}$
 - $p \in P = \{ \text{betSoFar, remainingChips, startChips} \}$
- **Actions** — **A**: $A = \{ \text{CHECK, RAISE, FOLD, ALL IN} \}$
- **Transition Model** — **result(s, a)** $\rightarrow s'$: $s' = s.\text{apply}(a).\text{updateActionHistory}(a).\text{advanceGame}()$
- **Terminal Test**: $\text{isGameOver} = \text{currentState.activePlayers}() == 1 \parallel \text{currentState.bettingRound} == \text{FINAL_ROUND}$
- **Utility** — **u(s)**: $\text{player.remainingChips}(\text{currentState}) - \text{player.remainingChips}(\text{previousState})$

Expectiminimax

Expectiminimax is an adversarial tree search algorithm that combines minimax with expected value nodes. Minimax is often used for algorithms that must consider the moves of an opponent. In this case, we must consider the moves of an opposing poker agent whose cards we do not know. Beyond not knowing the other player's cards, we also do not know what shared community cards will be played in the game. Expectiminimax deals with the stochastic nature of poker by calculating the expected utility of a given game state after reaching a particular depth. Previous applications of expected value functionality with poker include work done to estimate hand value (Neller et. al 2016) using expectimax. Expectiminimax builds on the following basic concepts:

Minimax Algorithm:

Minimax minimizes the possible loss for a worst-case scenario in zero-sum games.

```
def miniMax(state, depth):
    if depth == 0 or state.isGameOver():
        return state.expectedValue()
    if player.isMax():
        bestValue = -∞
        for each action in state.getLegalActions():
            v = miniMax(result(state, action), depth - 1)
            bestValue = max(bestValue, v)
        return bestValue
    else:
        bestValue = ∞
        for each action in state.getLegalActions():
            v = miniMax(result(state, action), depth - 1)
            bestValue = min(bestValue, v)
        return bestValue
```

Expectimax Algorithm

Expectimax maximizes the gains of a game state using expected value nodes.

```
def expectimax(state, depth):
    if depth == 0:
        return probability_a * (a_utility) +
            probability_b * (b_utility)
    else:
        values = []
        for each action in state.getLegalActions():
            successor = state.getSuccessor(action)
            value = expectimax(successor, depth - 1)
            values.append(value)
        return max(values)
```

Monte Carlo Tree Search (MCTS)

Monte Carlo Tree Search is a search algorithm used in decision-making processes, especially in the application of artificial intelligence in games. MCTS combines the general nature of sampling with the precision of tree search. Previous applications of this algorithm have shown its successful application in modern games such as Go, Chess, and Poker (Van den Broeck 2009). The adaptability and scalability of this algorithm make it an appropriate choice for Poker. Monte Carlo Tree Search builds on simpler algorithms and concepts:

Random Sampling — Monte Carlo Methods:

Monte Carlo methods utilize randomness to solve potentially deterministic problems. Monte Carlo methods

involve simulating random play-throughs of a game from a specified state and using the results of these simulations to estimate the desirability and utility of moves. Pseudocode for these sampling methods can be defined as follows:

```
def simulate(state):
    results = []
    for each action in state.getLegalActions():
        results[action] = simulateToTerminal(state, action)
    return determineOptimalAction(results)
```

Tree Search — Minimax Algorithm:

Tree search algorithms, such as the Minimax algorithm detailed earlier, navigate through potential moves and their counter-moves in games.

Monte Carlo Tree Search (MCTS) — Procedure:

Selection: In this stage, the algorithm traverses the game tree of actions based on the current game state. The selection process is guided by considerations that weigh the utility of choosing less explored areas of the decision tree (exploration) against deciding on moves that have previously had favorable results (exploitation). A common heuristic is the Upper Confidence Bound (UCB):

$$UCB = \overline{R}_j + C\sqrt{\frac{\ln(n)}{n_j}}$$

- $\overline{R}_j$ is the average reward of the node j
- n is the total number of simulations that have been run from node j
- n_j is the number of simulations that have been run from node j
- C is a constant that determines the trade-off between exploration and exploitation. A higher value encourages more exploration.

Expansion: When the algorithm reaches a leaf node in the tree, the expansion state adds new children nodes representing possible future actions or opponent decisions. This could mean adding nodes for different betting actions or responses to an opponent's move. This stage explores the numerous possible ways a poker hand could play out.

Simulation: In the simulation phase, the algorithm plays through the hand from the current state to a terminal state using a default algorithm. In this project, we used random play. This step in the algorithm estimates the potential

value of an action without the computational demands of an exhaustive search.

Backpropagation: The final stage of the algorithm updates the game tree with the results from the simulation. This means adjusting the values of nodes based on the wins and losses of the simulated hands. This step enhances the understanding of which actions are likely to be successful which can lead to better decision-making in future game states. (Browne et al. 2012).

Deep Q-Learning

Deep Q-Learning is an advanced reinforcement learning technique that combines the principles of Q-Learning with deep neural networks. Its most notable domains of use include Atari video games and robotics (for learning tasks without explicit rules). To understand Deep Q-Learning, it's essential to first understand the following concepts it builds on.

Reinforcement Learning (RL)

Reinforcement learning is a discipline of machine learning where an agent learns to make decisions by performing actions in an environment to achieve the specified goal. The agent receives rewards as feedback for its actions, guiding it toward the desired behavior (Mnih et al. 2015).

Q-Learning

Q-Learning is a simpler yet fundamental algorithm in reinforcement learning. At the heart of this method is the Q-Table which stores Q-values that estimate the equation (Q) of a specific action taken in a given state. The main idea is to update this table to reflect the expected utility of taking an action in a state. The update rule, derived from the Bellman equation, and pseudocode for Q-Learning are defined below.

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- α is the learning rate
- γ is the discount factor
- r is the reward
- s' is the new state.

```
def qLearning(state, alpha, gamma, epsilon, episodes):
    Q = getQTable(state)
    for episode in range(episodes):
        complete = False
```

```

while not complete:
    if random.uniform(0, 1) < epsilon:
        action = state.getRandomAction()
    else:
        action = argmax(Q[state])
    newState, reward, complete = state.apply(action)
    Q[state, action] += alpha * (reward + gamma *
                                max(Q[newState])
                                - Q[state, action])
    state = newState

```

Neural Networks

Inspired by the human brain, neural networks are a series of algorithms that recognize underlying relationships in a set of data through a process that is similar to the way the human brain operates.

Deep Q-Learning

Deep Q-Learning integrates Q-Learning with deep neural networks. Instead of using a Q-table, it utilizes a neural network, a Deep Q-Network (DQN), to approximate Q-values for each action in a given state. This is useful in situations with large state or action spaces where using a Q-table would be impractical. The Deep Q-Network is trained to minimize the loss between the predicted Q-values and the target Q-values that are estimated from the Bellman equation. The loss function is defined below (Mnih et al. 2015).

$$L(\theta) = \mathbb{E} [(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2]$$

- θ are the parameters of the DQN
- θ^- are the parameters of a target network, a copy of the DQN that is less frequently updated
- $\mathbb{E}$ is the expectation

Related Work

The evolution of artificial intelligence agents in poker has seen significant developments via the application of algorithms like Monte Carlo Tree Search, Expectiminimax, and Deep Q-Learning. Each of these models offers unique strengths in handling the complexity of poker. Previous works have shown how different approaches can be adjusted to handle the specific challenges of poker.

Studies utilizing MCTS have demonstrated its strength in managing poker's large decision spaces and unpredictability. Research findings highlight the

algorithm's ability to adapt as it navigates through different game scenarios, making it an effective tool in situations where exhaustive exploration of all possible actions is impractical. The study conducted by Browne et al. (2012) illustrates how MCTS can dynamically adjust strategies in response to the game as it unfolds, which is an essential feature in the dynamic context of poker (Browne et al. 2012).

Expectiminimax, on the other hand, focuses on its structured approach to making decisions in uncertain environments. This algorithm has been shown to effectively balance the adversarial nature of competition between players with the randomness of card dealing. A study by Russell and Norvig (2016) emphasizes expectiminimax's capability to integrate the probabilistic outcomes of poker, consequently helping in strategic planning against opponents and in environments influenced by chance (Russell and Norvig 2016).

The application of Deep Q-Learning in poker, as explored in research by Mnih et al. (2015), has highlighted its potential for long-term strategic learning. The study shows how through interaction with the game environment and feedback mechanisms, Deep Q-Learning can evolve complex strategies over time (Mnih et al. 2015). Blazejewski (2019) emphasizes the adaptability of Deep Q-Learning in handling the high-dimensional state spaces typical in poker, offering scalability and effectiveness in complex game environments. Blazejewski simplifies the reward structure by focusing on head's-up push-or-fold poker, wherein only two actions are possible, demonstrating how creative interpretation of the environment can allow for a more focused and effective application of the algorithm.

Other methods like Reinforcement Learning (RL) and Bayesian Methods could be applied to poker. Reinforcement Learning (RL), specifically policy gradient methods like REINFORCE and Actor-Critic, can be applied in poker for its ability to learn optimal policies directly from the game's feedback. These methods excel in environments where the agent needs to balance immediate rewards with a long-term strategy, a crucial aspect of poker where decisions have both immediate and extended consequences on the game's outcome. The main advantage of Reinforcement Learning lies in its ability to continuously learn and adapt, allowing the agent to develop and refine strategies through extensive interaction with the game environment. This continuous learning feature is relevant in poker, where the game dynamics

often change, and strategies need constant updating (Sutton and Barto 2018).

Bayesian methods could also be applied to poker. They provide a probabilistic approach to decision-making and can manage the game's inherent stochasticity and partially observable nature. Applying Bayesian networks could allow the agent to make informed inferences about an opponent's hands and strategies, leading to more meticulous and probabilistically valid decisions (Koller and Friedman 2009).

Despite the benefits that these alternate methods could provide, MCTS, Expectiminimax, and Deep Q-Learning are often preferred in poker due to their overall alignment with the game's demands.

Project Description

We conducted a comparative analysis of three algorithms to understand what the best approach to poker is. Our goal was to analyze how often, and with what pot size, three different poker agents would win a poker game. We can formally define our objective as follows

$$\max_{\text{wins, winnings}} P(\text{hand, community, deal})$$

Expectiminimax

Heuristic

To complete expectiminimax, we had to consider heuristics. Specifically, we needed to define some evaluation function to calculate the expected value of a given terminal node, defined as the estimated winnings of a poker game given any state-action pair.

The first step of calculating this expected value requires an evaluation of a given *agentHand*, based on any shared *communityCards*. This step requires considering all possible combinations of five out of seven cards. Note that at the game's start, when *agentHand* is of length two and *communityCards* is of length 0, the best five out of seven cards will always be a royal flush. For this reason, the evaluation function prompts the agent to always call when *communityCards* is of length 0.

Once we have a numerical value to represent an agent's hand and community cards, we can determine the probability of the agent winning. From here, we can estimate the expected pot winnings for an action based on our probability of winning, pot size, raise/call amount, amount bet so far, and starting chips. We can represent this evaluation function with the following pseudocode.

```
def evalGameState(state):
    value = estimateHand(state.agentHand,
                        state.communityCards, state.deck)
    probabilityWin = value / 100
    if state.lastAction == 'FOLD'
        return 0 - state.agent.betSoFar
    else if state.last_action == 'RAISE'
        numToWin = potSize + ( len(players) *
                             state.actionHistory.raiseAmount )
        numToLose = 0 - state.agent.betSoFar -
                    state.actionHistory.raiseAmount
        return probabilityWin * numToWin +
                (1-probabilityWin) * numToLose
    else if state.lastAction == 'CALL'
        numToWin = potSize +
                state.actionHistory.callAmount
        numToLose = 0 - state.agent.betSoFar -
                state.actionHistory.callAmount
        return probabilityWin * numToWin +
                (1-probabilityWin) * numToLose
    else if state.lastAction == 'ALL IN'
        numToWin = 2 * state.agent.startingChips
        numToLose = - 2 * state.agent.startingChips
        return probabilityWin * numToWin +
                (1-probabilityWin) * numToLose
```

Additional Details

Expectiminimax is called whenever it is the agent's turn to bet. This algorithm loops through all available legal actions (composed of FOLD, RAISE, CALL, or ALL IN) and recursively iterates to a depth of three to maximize the expected pot winnings of the agent while minimizing the expected pot winnings of the opponent. To account for the partially observable nature of not knowing the opponent's hand, expectiminimax randomly deals two cards from the deck to represent the opponent's hand.

Monte Carlo Tree Search (MCTS)

Implementing Monte Carlo Tree Search requires the definition of tree and leaf nodes which represent a complex branching factor. To begin, an MCTS agent is initialized when it is the agent's turn. The agent iterates over a set number of simulations, repeating the steps for selection, expansion, simulation, and backpropagation until some final tree is created and the best action can be selected.

In our specific implementation, we used 50 simulations, as this number seemed to be the maximum number of simulations we could run without significantly slowing down our run time. Even running 50 simulations, our expected run time for a single poker hand was approximately five minutes. From the root node of each simulation, the next most promising node, when our root is fully expanded, was selected using Upper Confidence Bound (UCB) across all available child nodes. Referencing the earlier definition of UCB, this utility was defined as follows.

$$UCB = \frac{w_j}{n_j} + 1.41\sqrt{\frac{\ln(n)}{n_j}}$$

- w_j is the number of wins on node j
- n_j is the number of visits to node j
- n is the number of visits to the parent of node j
- Our exploration constant is 1.41

Having selected a start node, we expand our MCTS tree by a singular node. In this expansion stage, we choose an action to execute from our list of untried actions. From here, we generate a successor state and add it to our tree. Once we have a tree with an expanded node, we begin the simulation stage. This simulation starts from our most recently added expansion node and simulates a poker game to completion. The game is simulated by switching between the agent and the opponent through all remaining phases of the game. Once the game has concluded, we check if the agent has won. To backpropagate the tree we start with the current node and increment its number of visits and win count (if applicable), traversing up the tree until we reach our root node.

After we have iterated across all fifty simulations, we select the best node as the immediate child node of our root node with the highest win rate. Since each child node references a successor state of the root node given some action, the child node with the highest win rate is the best node to select. This action is then fed into our poker implementation, and this process is repeated every time the agent must decide the end of the game.

Deep Q-Learning

Neural Networks

Our implementation of deep Q-Learning began by defining a neural network that we would use to estimate

q-values. For this network, we referenced the *tensorflow* and *numpy* Python libraries. Our defined neural network had an input layer, a hidden layer, and an output layer. Our input was defined as an array that represented the strength of *agentHand* and *communityCards*, *potSize*, and the *remainingChips* of our player agent. Hand value was calculated by generating possible 5-card hands from the given cards, assigning a numerical value/score to each possible hand based on its rank, and returning the average (if not all cards dealt) or highest (if all cards dealt) of these values. Pot size and player chips were both fetched directly from the game state.

- $cardsArray = [h_0, h_1, h_2, h_3]$
 $h_0 = 1$ if $5000000000 \leq hand_value < 10000000000$,
else 0
 $h_1 = 1$ if $10000000000 \leq hand_value < 15000000000$,
else 0
 $h_2 = 1$ if $15000000000 \leq hand_value < 20000000000$,
else 0
 $h_3 = 1$ if $20000000000 \leq hand_value$, else 0
- $potSize = [p_0, p_1, p_2, p_3]$
 $p_0 = 1$ if $50000 \leq pots < 100000$, else 0
 $p_1 = 1$ if $100000 \leq pots < 150000$, else 0
 $p_2 = 1$ if $150000 \leq pots < 200000$, else 0
 $p_3 = 1$ if $200000 \leq pots$, else 0
- $ourChips = [c_0, c_1, c_2, c_3]$
 $c_0 = 1$ if $50000 \leq chips < 100000$, else 0
 $c_1 = 1$ if $100000 \leq chips < 150000$, else 0
 $c_2 = 1$ if $150000 \leq chips < 200000$, else 0
 $c_3 = 1$ if $200000 \leq chips$, else 0
- Input array = $cardsArray + potSize + ourChips = [h_0, h_1, h_2, h_3, p_0, p_1, p_2, p_3, c_0, c_1, c_2, c_3]$

The output layer was defined as all possible actions from a given state and some corresponding q-value, initialized to a random small number. Based on the existing work found in *Applying Deep Reinforcement Learning to Poker* (Blazejewski 2019) and *Approximating Poker Probabilities with Deep Learning* (Da Silva 2018), we chose to implement a rectified linear unit (ReLU) activation function and the Adam optimization to minimize Mean Squared Error loss function.

Reinforcement Learning

Having defined a neural network model with an input layer, output layer, activation function, and loss function, our next step was to track and update q-values for state-action pairs after each episode. We initialized q-values to random small numbers at the start of the game. For every instance that it was the agent’s turn to take an action, we selected the action with the best q-value. Following the implementation of the action, we then tracked the updated game state and reward. We defined reward deterministically as 0 when the game was not over and as equal to the pot size otherwise. Upon updating some stored memory with new state-action-state-reward tuples, we selected a batch of size 32 and updated our neural network (Sutton and Barto 2018).

Experiments

Implementation

For our general implementation of this project, we used basic poker functionality as implemented in the git repo Command-Line-Poker (Elery 2022). This functionality includes a command-line run poker implementation, definitions of poker-specific classes (deck, card, player, etc), a random poker opponent to play against, and the definition of a game.

Across all three algorithms, we defined a PokerGameState class which was passed in a game (from our utility repo). This game state contains all fields outlined in our problem formalization of a poker state. We also defined three agents, one for each algorithm.

We ran each algorithm in 2-player poker against a simple bot. Our implementations of Expectiminimax and MCTS both played 100 hands against each opponent for a total of 300 hands against three different opponents. To account for learning, our implementation of Deep Q-Learning played 1000 hands against each opponent. The decision-making probabilities for each opposing bot are listed in Table 1.

Action	Safe Bot	Risky Bot	Random Bot
FOLD	0.70	0.40	0.33
CHECK/CALL	0.20	0.50	0.33
BET/RAISE	0.10	0.10	0.33

Table 1: Decision-Making Probabilities for Poker Bots

We opted to test our algorithm implementations against simple bots, as opposed to each other, for several reasons. Firstly, simple bots serve as a benchmark, allowing us to judge how well our artificial intelligence agents perform against basic gameplay standards. This comparison allows us to validate that our algorithms outperform these pre-defined strategies. Second, simple bots offer a stable and predictable testing environment. Their predetermined, less nuanced behavior provides a clear basis for analyzing and understanding the specific strengths and weaknesses of our algorithms. This controlled setting is useful for assessing how our algorithms adjust to different playing styles, such as consistently aggressive or passive approaches. Finally, the metrics collected from matches against these basic bots are simple to analyze. This clarity helps us identify and quantify the developments in the decision-making capabilities and their ability to adapt their strategies.

We defined each game, from the initial card dealing to the moment a player won the pot, as one hand. To remain consistent across all games, we established specific presets. Each player started with 100,000 chips, and the big blind was set at 2,000 chips. The starting chips for each player were reset to 100,000 at the beginning of every hand. This approach ensured an even playing field for each new game, creating a fair and standardized comparison of performance across various hands.

During matchups between our algorithms and the bots, we focused on three key metrics to evaluate their performance. The first of these metrics was wins, the number of hands won by the algorithm out of the 100 hands played. The second metric, winnings, quantified the number of chips won by the algorithm in each hand, with the potential range of winnings falling between -100,000 and 100,000, as a consequence of the starting chip amount. Our last metric total net winnings, summed the total chip count won by the algorithm across all 100 hands.

These metrics provided a comprehensive view of the algorithm’s effectiveness in varied game scenarios, both in terms of win rate and actual chips won. A high win rate and high total winnings signify well-performing algorithms. A relatively high ratio of wins to winnings suggests a safe algorithm that prioritizes winning the game and folding when necessary, while a lower ratio indicates a risky algorithm that is bold enough to call or bet.

Results

Our algorithms outperformed their opponents in all matchups, as seen in Table 2. The high probability (0.33-0.70) of the simple bots folding for any given action, thus forfeiting the hand, is a likely contributor to the overall stellar performance of our algorithms.

Algorithm	Metric	Safe Bot	Risky Bot	Random Bot
Expectiminimax (100 trials/bot)	Win Rate	79%	78%	79%
	Net Winnings	672,000 chips	954,000 chips	774,000 chips
MCTS (100 trials/bot)	Win Rate	93%	91%	91%
	Net Winnings	836,000 chips	1,093,000 chips	1,023,000 chips
Deep Q-Learning (1000 trials/bot)	Win Rate	53.8%	51.2%	55.9%
	Net Winnings	2,876,300 chips	2,568,000 chips	2,480,000 chips

Table 2: Poker Matchup Results

The spread of winnings by hand for each algorithm-bot matchup is laid out in Figures 1-9 below. Each point marks the winnings of the algorithm in one hand of play. So, if some algorithm won a 20,000-chip pot containing 10,000 of its chips and 10,000 of the bot's chips in a hand, its winnings for the hand would be 10,000 chips; if it lost that pot, its winnings would be -10,000 chips.

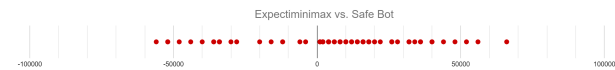


Figure 1: Expectiminimax Winnings per Hand vs. Safe Bot



Figure 2: Expectiminimax Winnings per Hand vs. Risky Bot

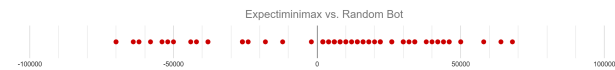


Figure 3: Expectiminimax Winnings per Hand vs. Random Bot

Overall, we found that our implementation of the expectiminimax agent demonstrated notable efficiency as well as a high win rate. A crucial aspect of its decision was its quick decision-making process, specifically in resolving game states with identical hands. This algorithm primarily focused on expected values and favored options with the least negative image. This is a practical choice given the scarcity of high-value hands in poker. We set the search depth to a value of three, thus balancing between computational speed and accuracy. It's worth noting that increasing this depth could future enhance the precision of the agent but at the expense of speed and efficiency. When evaluating potential hands, our implementation considered different, unordered, combinations of cards. We treated similar but suit-specific hands, like a royal flush in spades versus one in hearts, as distinct entities. This not only streamlines the computation but also aligns with the distinct rankings of poker hands.

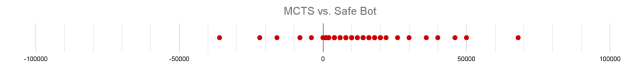


Figure 4: MCTS Winnings per Hand vs. Safe Bot

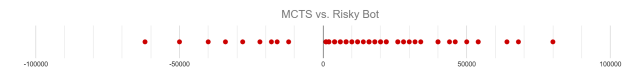


Figure 5: MCTS Winnings per Hand vs. Risky Bot

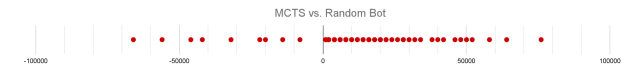


Figure 6: MCTS Winnings per Hand vs. Random Bot

Our implementation of Monte Carlo Tree Search performed similarly well against all three bots. The Safe Bot is highly likely (0.70 probability) to fold for any given action and forfeit the hand, resulting in a slightly higher (93%) win rate for MCTS. The safe nature of this bot also resulted in slightly lower average and total winnings for MCTS, as the Safe Bot was likely to fold earlier in the game without betting enough to significantly increase the pot.

If we look at Monte Carlo Tree Search as a whole, there are two main factors to consider: its high win rate, and its slow running time. Both of these features are related to the very high branching factor of MCTS. Since we chose to set our number of simulations to 50, and we iterate through simulations every time it is our agent's turn to bet, the inherent scale for our implementation is very large. Given that the branching factor of MCTS is

exponential, we found that the run of a singular poker hand, where no one folds, from start to finish took approximately five minutes. Expectiminimax comparatively took much less time because it calculates the expected value of the agent's hand but does not simulate gameplay. However, since Monte Carlo Tree Search has a large branching factor that executes games from start to finish, it has a high probability of winning.

Another reason that Monte Carlo Tree Search may perform well here has to do with tiebreakers. When a fully expanded parent node uses a win rate to determine the best child node, it will occasionally have three child nodes with the same win rate (for example, if the player does not have a good hand and loses in all simulations). In these cases, we chose to prioritize calling over raising and raising over folding. This means that our agent is inherently less likely to fold (or automatically lose the game) than it is to call or raise, further contributing to a high win rate.

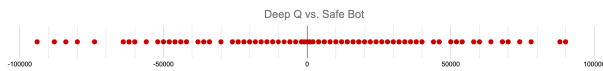


Figure 7: Deep Q-Learning Winnings per Hand vs. Safe Bot

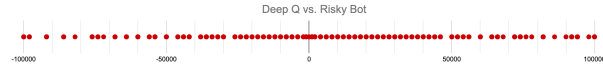


Figure 8: Deep Q-Learning Winnings per Hand vs. Risky Bot



Figure 9: Deep Q-Learning Winnings per Hand vs. Random Bot

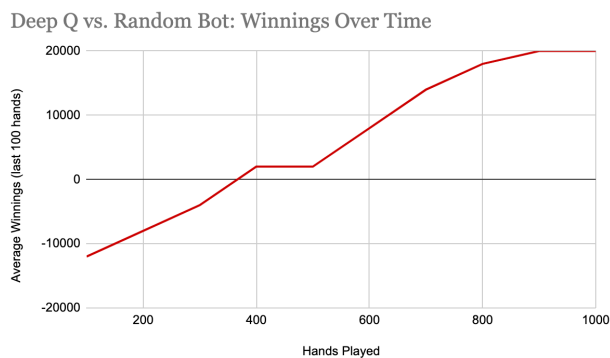


Figure 10: Deep Q-Learning Average Winnings Over Time

In our experiments with Deep Q-Learning, we observed an interesting evolution in performance. Initially, the agent's performance was surprisingly terrible. This phase of its learning, however, was not indicative of the inadequacy of the algorithm but was instead an indicator of a crucial step in its learning process. As the agent continued to engage in more episodes and hands, its capabilities improved dramatically, evolving from poor to impressive performance levels. This improvement underscored the algorithm's requirement for a significant number of episodes to achieve consistent behavior and increased win rates. The overall operation and completion of the algorithm were efficient due to the use of external libraries for the implementation of the neural network.

We used several constants to define our Deep Q-Learning agent, all of which may influence our final results. These are batch size, exploration rate, and learning rate. These constants were chosen arbitrarily but referenced the work completed by Blacjewski (2019) and Da Silva (2018). In future iterations of this project, we could experiment with how these variables affect the progress of Deep Q-Learning. However, since our project focused on Deep Q-Learning as a comparison to other algorithms, we chose to keep these variables constant. One particular choice we made surrounding reward utilities was to set the reward to 0 when an episode is complete and the game is not yet complete. We define an episode here as the result of a single betting move. When the game is complete, we set the reward to the net winnings of the agent. Since we did not discount rewards, an alternative to this approach would be to use discounting or negative rewards, as another way to prevent the agent from prolonging the betting period.

Compared to Expectiminimax and Monte Carlo Tree Search, Deep Q-Learning uses reinforcement learning. This means that, unlike the other two algorithms, Deep Q-Learning does not consistently perform the same functionality. Rather, this algorithm began by making near-random action choices based on arbitrary initial q values. As the agent experienced multiple games, these values updated, allowing for improved decision-making. This means that while the average percent of games won by Deep Q-Learning averaged a little above 50% across three bots, by its final

episodes the agent was able to consistently make game-winning decisions. This is a massive improvement over Monte Carlo Tree Search, which we found very slow, and Expectiminimax, which is quick but does not consistently win.

Conclusion

In this work, we developed three distinct AI agents to play Texas Hold'em poker, utilizing Expectiminimax, Monte Carlo Tree Search, and Deep Q-Learning. Our results showed that each algorithm had unique strengths, with Monte Carlo Tree Search showing the best adaptability and Deep-Q Learning excelling in long-term strategy development. We learned about the capabilities of artificial intelligence in handling probabilistic scenarios and strategy construction in stochastic environments.

The biggest takeaways of this project were the significant insights on the role of heuristics in AI decision-making for poker. The choice of heuristics was pivotal in improving the efficiency and effectiveness of the algorithms. Within the contexts of Expectiminimax and Monte Carlo Tree Search, carefully defined heuristics resulted in more accurate decision-making and better risk assessment. In Deep-Q Learning, the integration of heuristic-based rewards guided the agent toward more strategic, long-term play. These findings highlight the importance of heuristic selection in an agent's potential to navigate complex, partially observable, and stochastic environments like poker, suggesting broader implications for applications of artificial intelligence in strategic decision-making.

References

- Blazejewski, O. (2019, January 9). *Applying deep reinforcement learning to poker*. Adaltas RSS. <https://www.adaltas.com/en/2019/01/09/applying-deep-reinforcement-learning-poker/>
- Browne, C. B., Powley, E., Whitehouse, D., Lucas, S. M., Cowling, P. I., Rohlfshagen, P., ... & Colton, S. (2012). *A survey of Monte Carlo tree search methods*. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1), 1-43.
- Brunkhorst, A., Keesara, S., Onyejiaka, N. (2023). *Artificial Intelligence in Poker: A Comparative Analysis of Decision-Making Using Search Algorithms* <https://github.com/CS-5100-Poker/poker-cs-5100>
- Catral, R., Oppacher, F. (2007). *Poker Hand*. UCI Machine Learning Repository. <https://doi.org/10.24432/C5KW38>
- Da Silva, Brandon. (2018). *Approximating Poker Probabilities with Deep Learning*.
- Elery, Q. (2022). *Command-Line-Poker: A collection of code and data for my research*. GitHub. <https://github.com/qelery/Command-Line-Poker>
- GeeksforGeeks. (2020). *Monte Carlo Tree Search (MCTS) in Machine Learning*. <https://www.geeksforgeeks.org/ml-monte-carlo-tree-search-mcts/>
- Haykin, S. (2009). "Neural Networks and Learning Machines." Pearson Education.
- Koller, D., & Friedman, N. (2009). *Probabilistic Graphical Models: Principles and Techniques*. MIT Press.
- Konstantinova, O. (2022). *Explainable AI: Explaining Poker with LIME*. Medium. <https://medium.com/@okonstantinova/explainable-ai-explaining-poker-with-lime-9db206c2d5e9>
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., ... & Petersen, S. (2015). *Human-level control through deep reinforcement learning*. *Nature*, 518(7540), 529-533.
- Neller, T., Messinger, C., & Yang, Z. (2016). *Learning and Using Hand Abstraction Values for Parameterized Poker Squares*. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1). <https://doi.org/10.1609/aaai.v30i1.9859>
- Russell, S. J., & Norvig, P. (2016). *Artificial Intelligence: A Modern Approach*. Pearson Education Limited.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press.
- Van den Broeck, G. (2009). *Monte-Carlo tree search for multi-player, no-limit Texas hold 'em poker Monte-Carlo tree search for multi-player*. Department of Computer Science, UCLA. <https://web.cs.ucla.edu/~guyvdb/slides/SIKS11.pdf>
- Van den Broeck, G., & Darwiche, A. (2009). *On the complexity and approximation of binary evidence in lifted inference*. *Proceedings of the 23rd Annual Conference on Neural Information Processing Systems (NIPS)*. <http://starai.cs.ucla.edu/papers/VdBACML09.pdf>